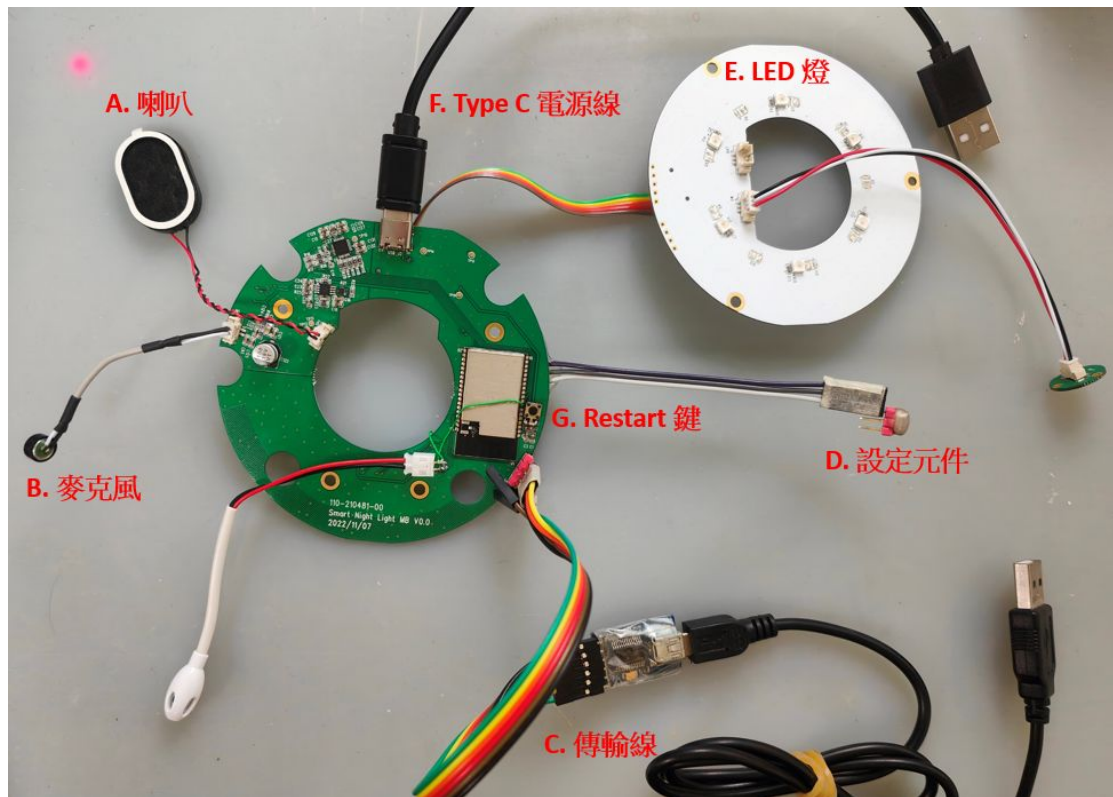
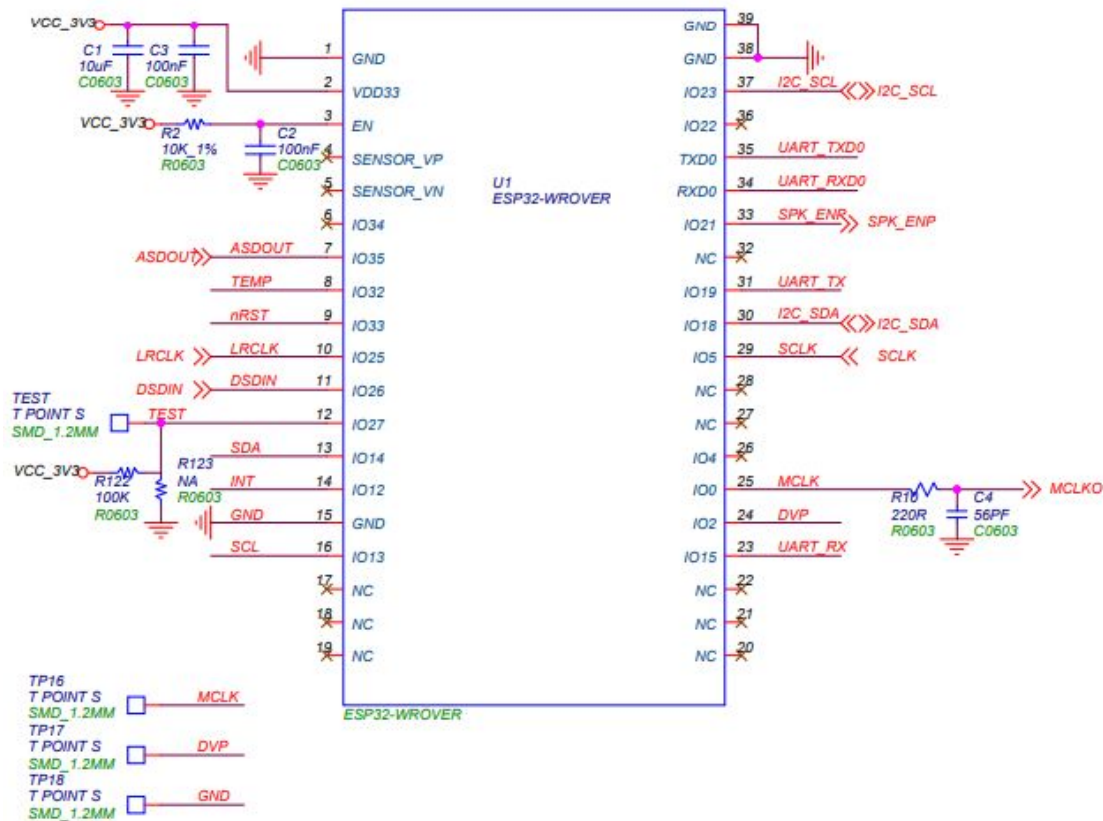


Smart Speaker Platform

平台主板



- A. 喇叭:機器輸出語音
- B. 麥克風:接收使用者指令
- C. 傳輸線:燒錄程式碼
- D. 設定元件:進入上傳模式
- E. LED燈:
- F. Type-C 電源線:提供電源
- G. Restart鍵:重啟機器

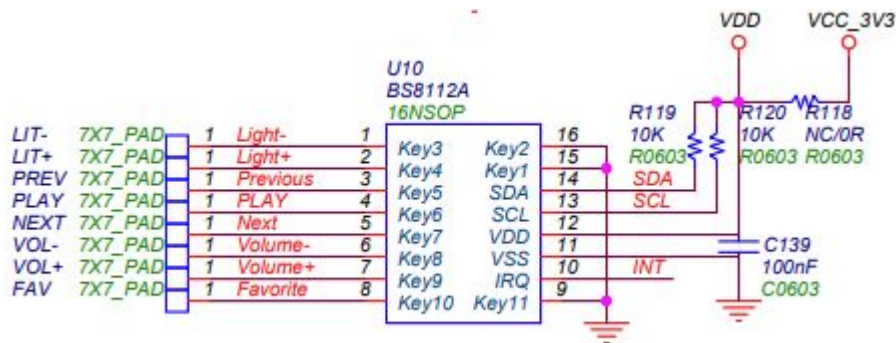


- 核心晶片:Espressif ESP32-D0WDQ6/ESP32-D0WD (雙核 240MHz MCU)。
- 儲存配置:
 - Flash:標準 4MB SPI Flash, 部分型號可達 16MB。
 - PSRAM:內建 4MB/8MB PSRAM。
- 無線技術:
 - Wi-Fi:802.11 b/g/n (最高 150 Mbps)。
 - 藍牙:Bluetooth 4.2 BR/EDR 與低功耗藍牙 (BLE)。
- 天線選擇:
 - PCB 板載天線 (如 ESP32-WROVER, ESP32-WROVER-E)。
 - IPEX 接頭外接天線 (如 ESP32-WROVER-I, ESP32-WROVER-IE)。
- 介面與外設:GPIO、UART、SPI、I2C、I2S、ADC、DAC、PWM、SD 卡接口、電容式觸控、霍爾傳感器。

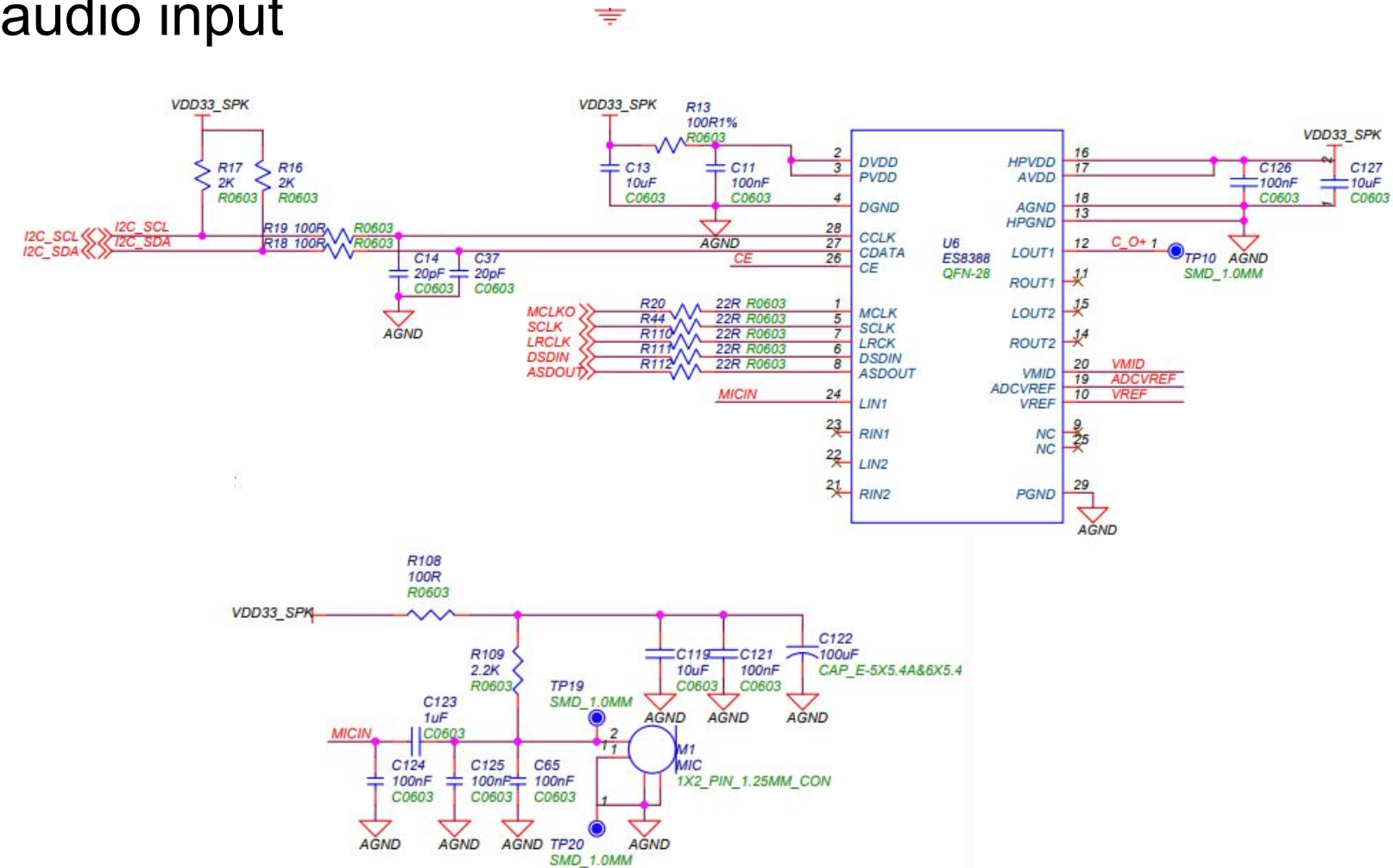
Touch IC

目前在原始碼中沒有它的驅動程式。

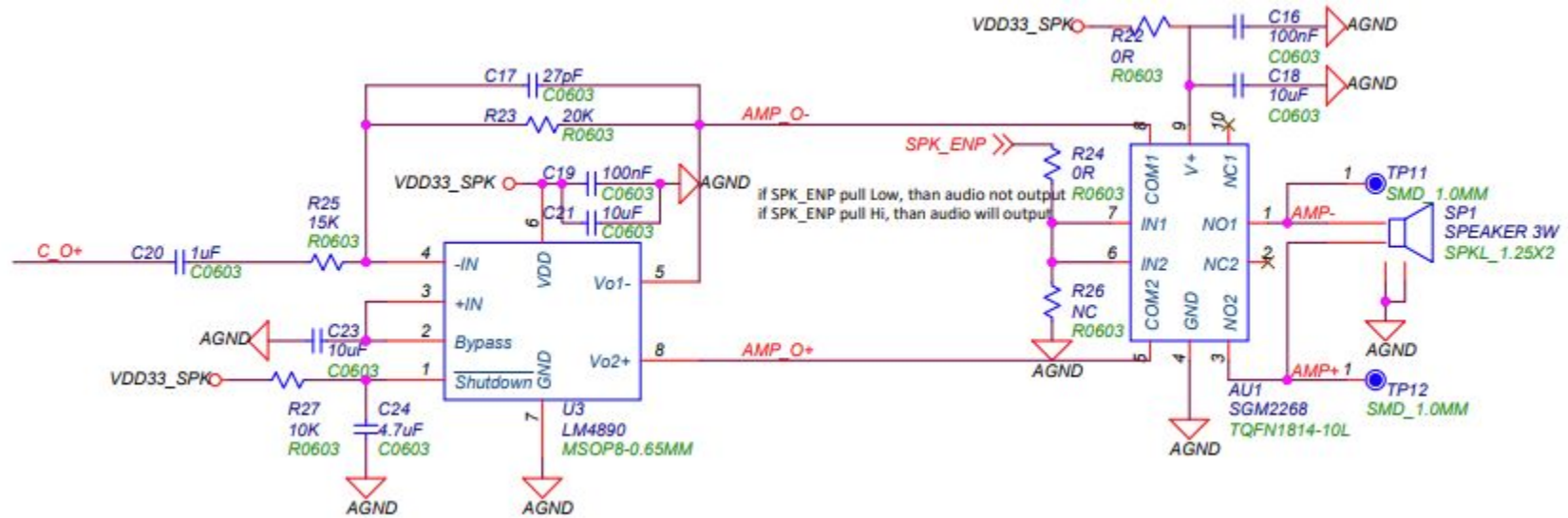
[bs8112 datasheet](#)



audio input



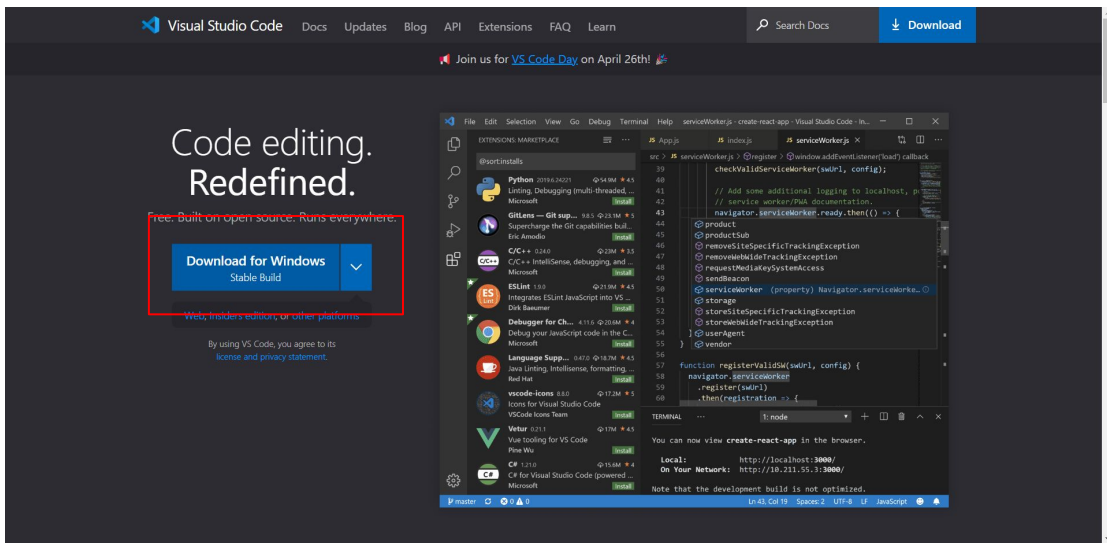
audio output



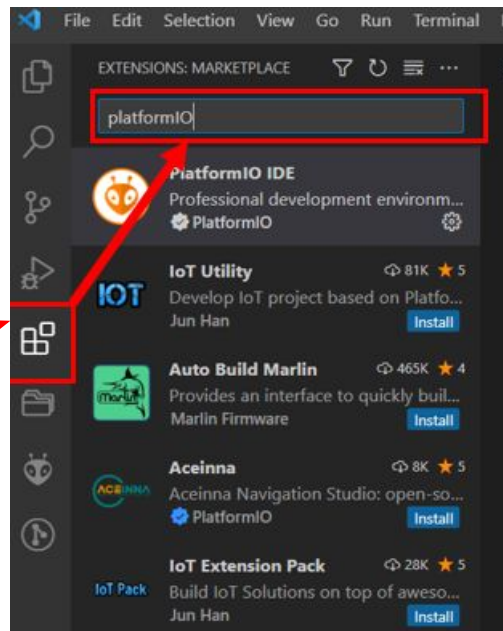
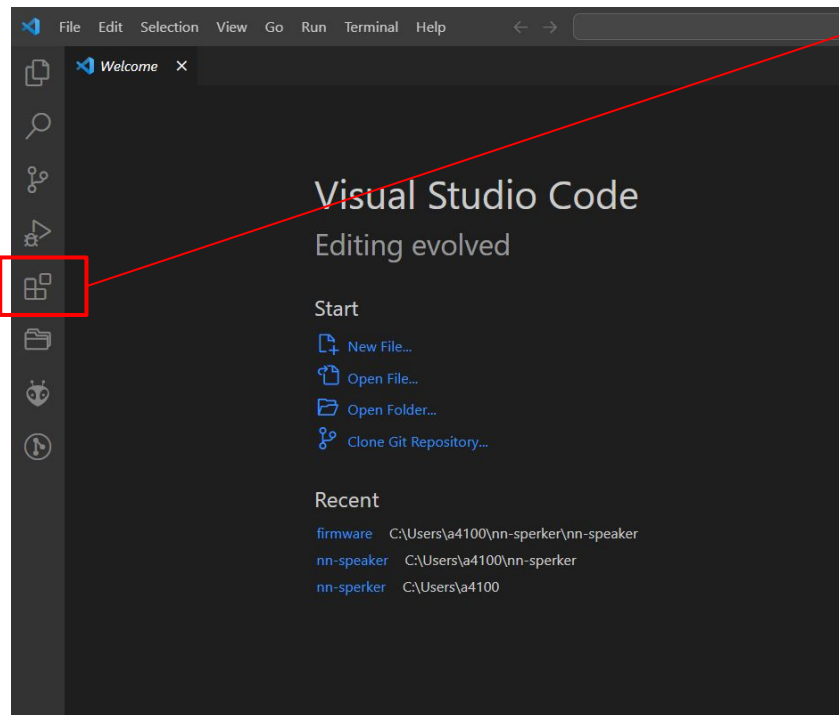
安裝作業環境-安裝 VSCode

<https://code.visualstudio.com/>

如使用雲端平台，只要遠端
remote-desktop 就可以了。VSCode
已預裝在桌面上



安裝 platform IO

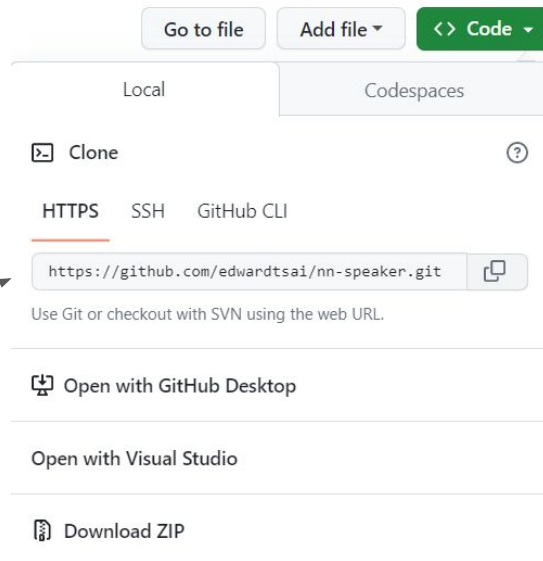
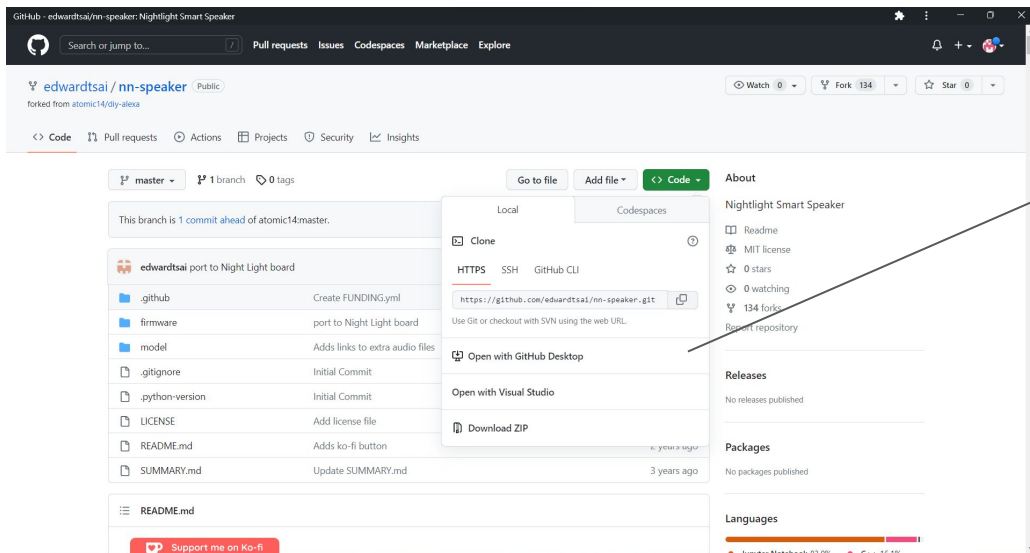
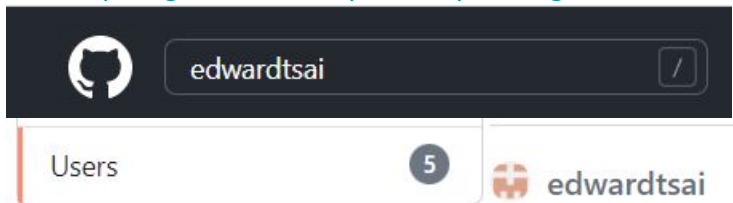


Platform IO 可以用來開發各式 MCU 的程式。

請選擇應用程式列表，然後使用 platformio 做關鍵字找出 platformIO 模組安裝。

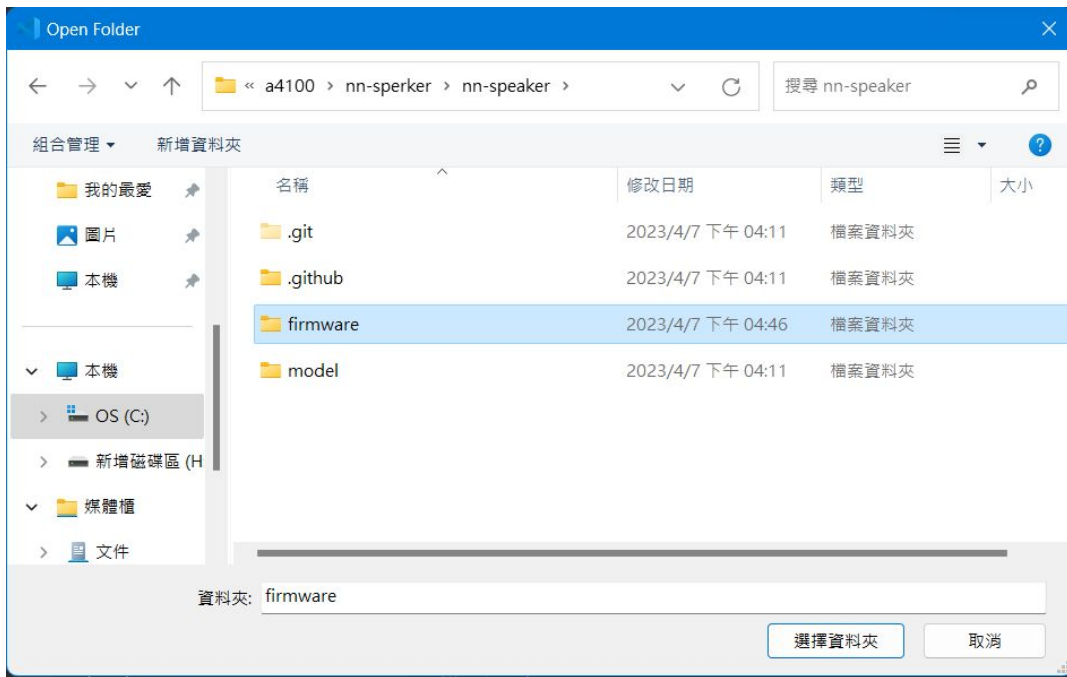
download nn-speaker

<https://github.com/wycc/nn-speaker.git>

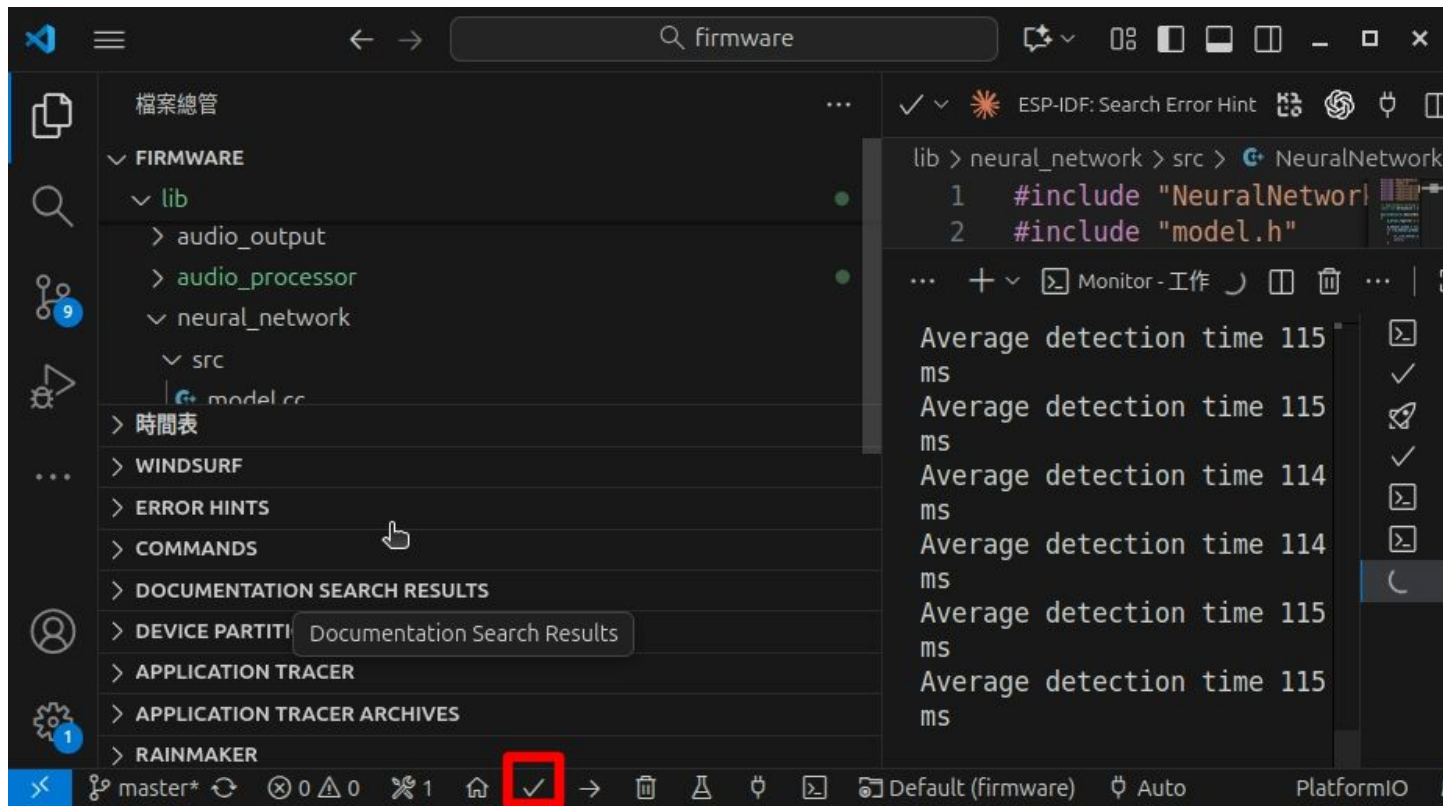


Open firmware folder

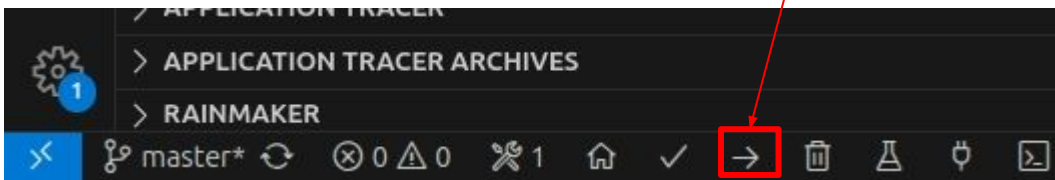
在Visual Studio Code裡開起nn-speaker 資料夾中的firmware資料夾



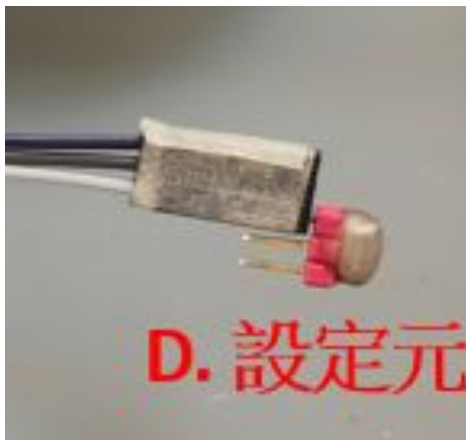
Compile the program



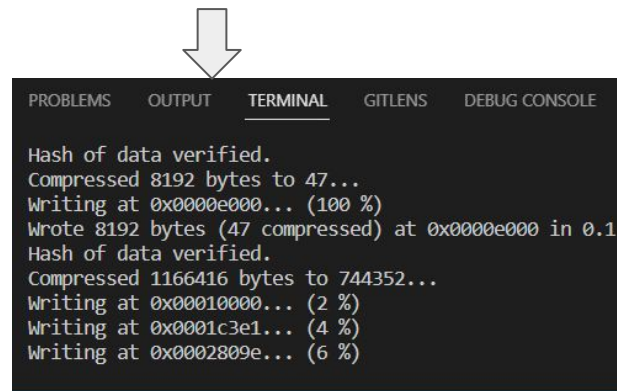
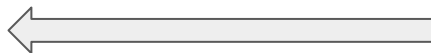
下載程式



進入下載模式前, 先將機器上3pin線短路在一起, 進入設定模式, 再重新上電, 回到 Platform IO按Upload, 就會開始上傳新的程式

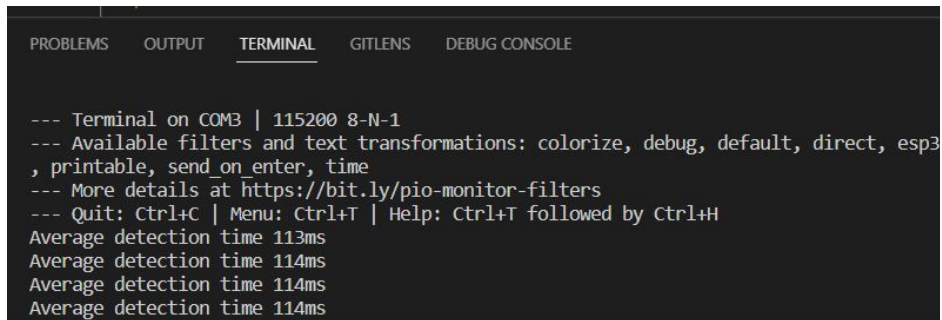
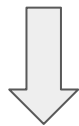
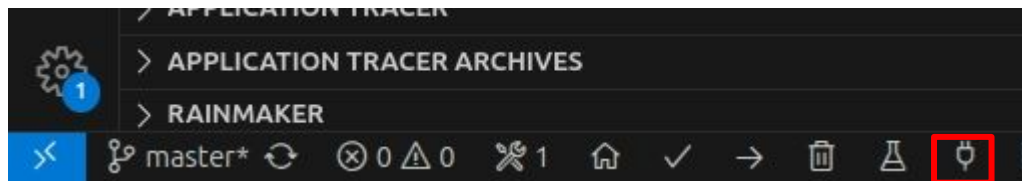
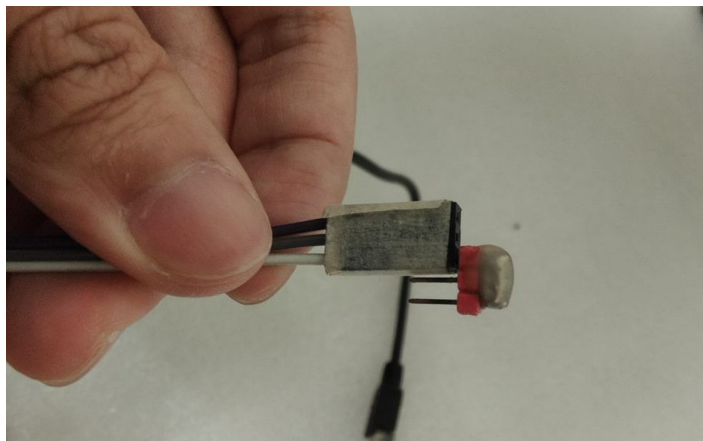


下載完後將 3pin移除



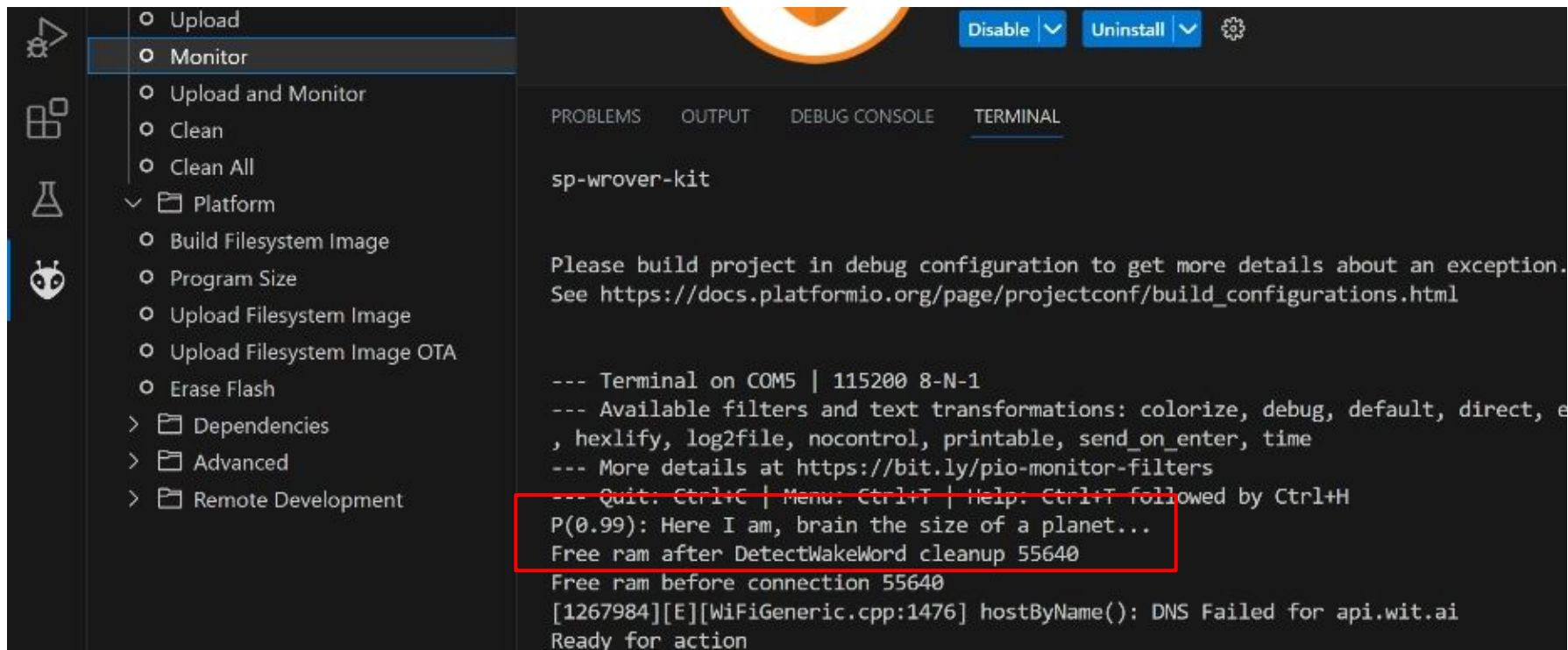
重啟系統

Upload 完成之後再將剛剛短路在一起的
3pin線分開, 再重新上電



測試

之後再開啟Platform IO 按下Monitor開始感測，說出Marvin，機器就會逼一聲。



Step 1: convert all audio files into spectrogram

- <https://github.com/edwardtsai/nn-speaker/blob/master/model/Generate%20Training%20Data.ipynb>
- Conver the audio into mel spetrogram take long time. We will do it once only.
- Therefore, we will save the files into
 - training_spectrogram.npz
 - validation_spectrogram.npz
 - test_spectrogram.npz

Step 2: Train the model

- <https://github.com/edwardtsai/nn-speaker/blob/master/model/Train%20Model.ipynb>
- trained.model
- fully_trained.model

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv_layer1 (Conv2D)	(None, 99, 43, 4)	40
max_pooling1 (MaxPooling2D)	(None, 49, 21, 4)	0
conv_layer2 (Conv2D)	(None, 49, 21, 4)	148
max_pooling2 (MaxPooling2D)	(None, 24, 10, 4)	0
flatten (Flatten)	(None, 960)	0
dropout (Dropout)	(None, 960)	0
hidden_layer1 (Dense)	(None, 40)	38440
output (Dense)	(None, 1)	41
=====		
Total params: 38,669		
Trainable params: 38,669		
Non-trainable params: 0		

Step 3: convert to tflite model

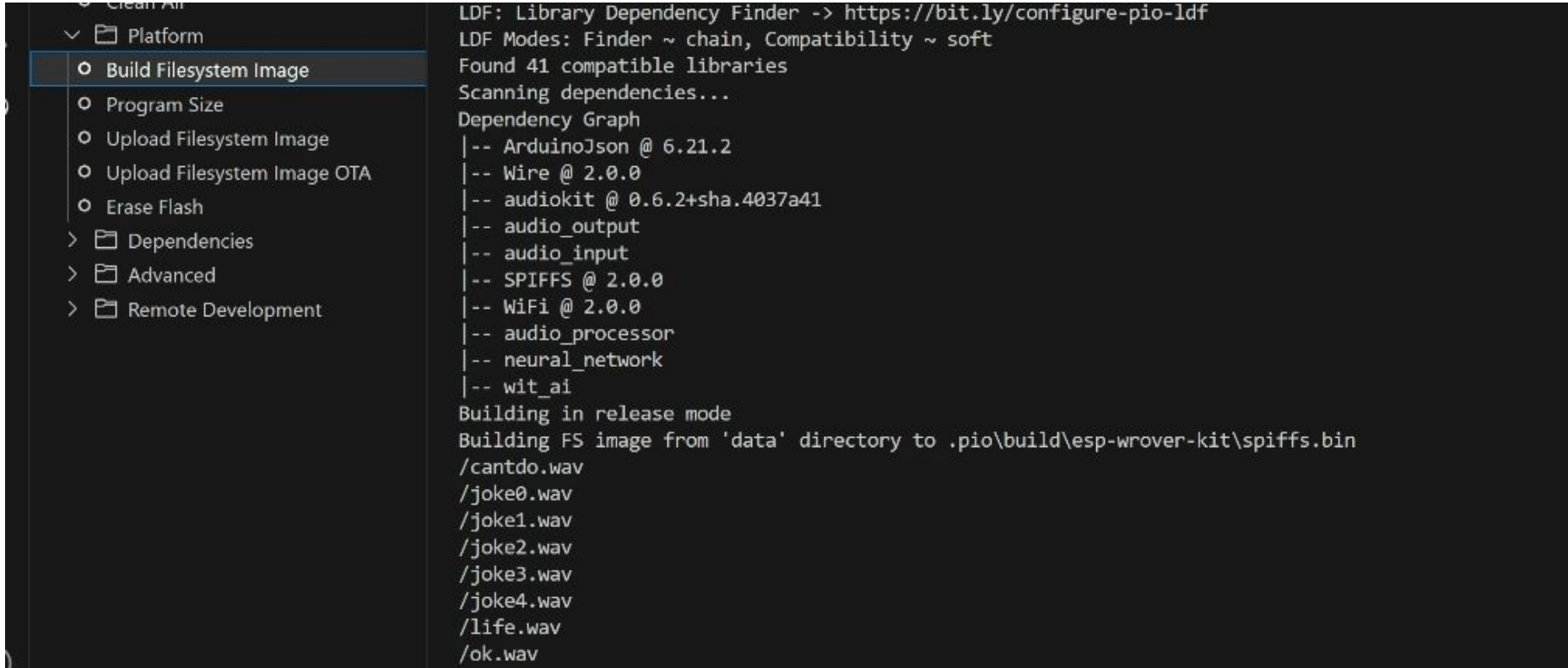
<https://github.com/edwardtsai/nn-speaker/blob/master/model/Convert%20Trained%20Model%20To%20TFLite.ipynb>

- Convert to tensorflow lite model
- convert the model into C source code so that we can link it into the firmware for the ESP32.

下載新的 firmware

請將之前訓練好的模型中 `model_data.cc` 取代原先 `nn-speaker` 中的同名檔案。重新編輯後下載。就可以測試新的關鑑字了。

Prepare filesystem



Arduino IDE interface showing the 'Build Filesystem Image' option selected in the Platform menu. The right pane displays the LDF (Library Dependency Finder) output.

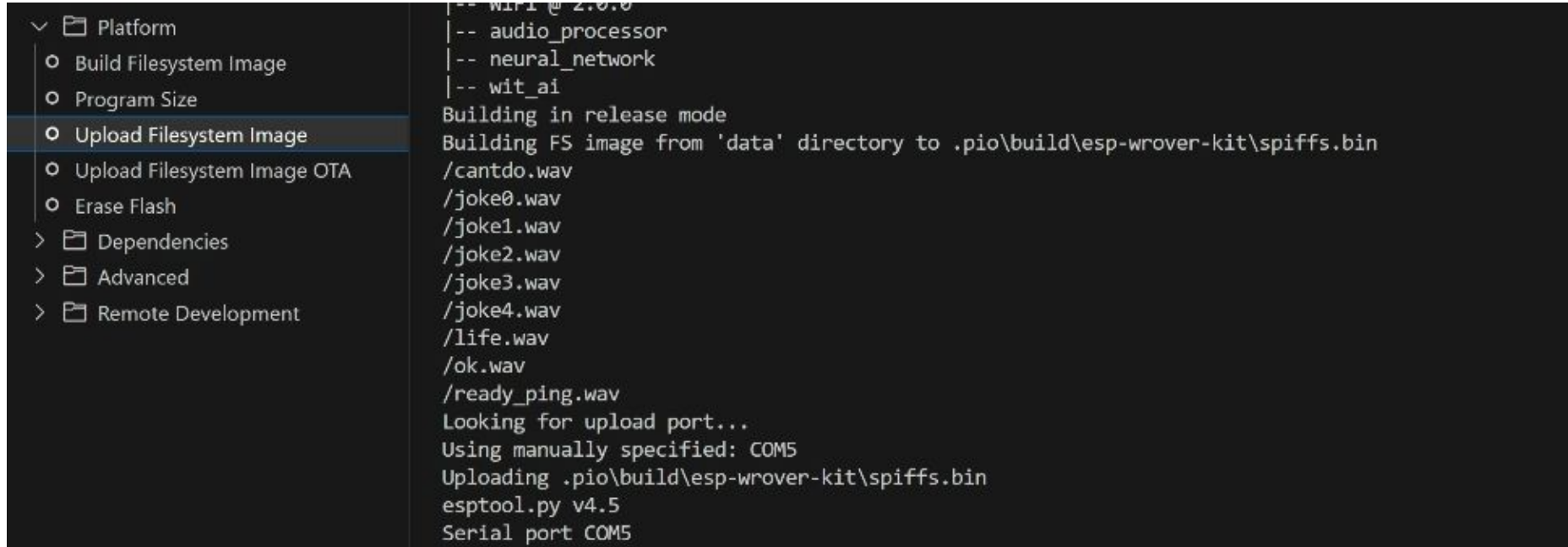
Platform Menu:

- Build Filesystem Image (selected)
- Program Size
- Upload Filesystem Image
- Upload Filesystem Image OTA
- Erase Flash
- Dependencies
- Advanced
- Remote Development

LDF Output:

```
LDF: Library Dependency Finder -> https://bit.ly/configure-pio-ldf
LDF Modes: Finder ~ chain, Compatibility ~ soft
Found 41 compatible libraries
Scanning dependencies...
Dependency Graph
|-- ArduinoJson @ 6.21.2
|-- Wire @ 2.0.0
|-- audiokit @ 0.6.2+sha.4037a41
|-- audio_output
|-- audio_input
|-- SPIFFS @ 2.0.0
|-- WiFi @ 2.0.0
|-- audio_processor
|-- neural_network
|-- wit_ai
Building in release mode
Building FS image from 'data' directory to .pio\build\esp-wrover-kit\spiffs.bin
/cantdo.wav
/joke0.wav
/joke1.wav
/joke2.wav
/joke3.wav
/joke4.wav
/life.wav
/ok.wav
```

download filesystem



```
Platform
├── Build Filesystem Image
├── Program Size
├── Upload Filesystem Image
├── Upload Filesystem Image OTA
├── Erase Flash
├── Dependencies
├── Advanced
├── Remote Development
└── ...

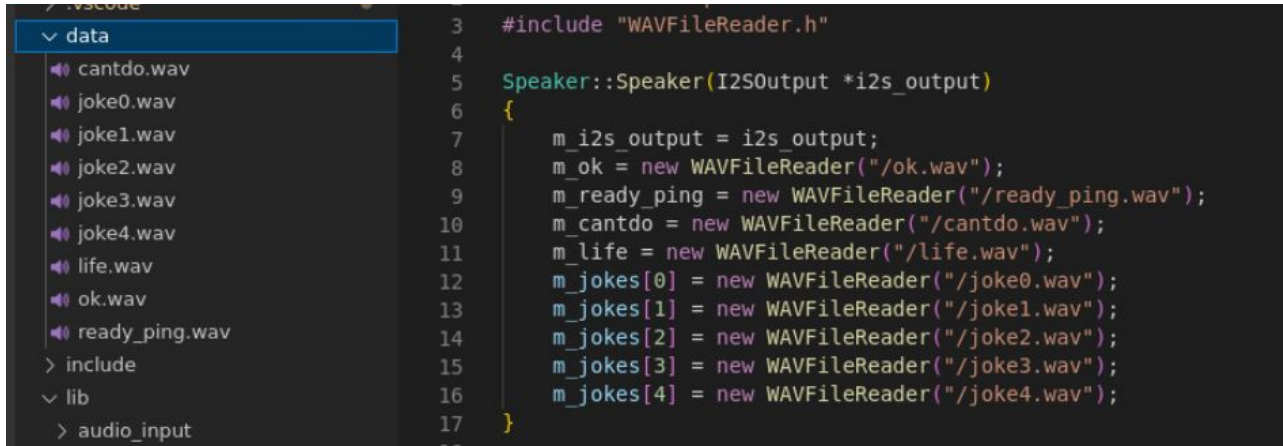
|-- WiFi @ 2.0.0
|-- audio_processor
|-- neural_network
|-- wit_ai
Building in release mode
Building FS image from 'data' directory to .pio\build\esp-wrover-kit\spiffs.bin
/cantdo.wav
/joke0.wav
/joke1.wav
/joke2.wav
/joke3.wav
/joke4.wav
/life.wav
/ok.wav
/ready_ping.wav
Looking for upload port...
Using manually specified: COM5
Uploading .pio\build\esp-wrover-kit\spiffs.bin
esptool.py v4.5
Serial port COM5
```

We only need to download once unless we change the files.

output sound effect

```
void RecogniseCommandState::enterState()
{
    // indicate that we are now recording audio
    m_indicator_light->setState(ON);
    m_speaker->playReady();
}
```

m_speaker can be used to output audio to the I2S speaker

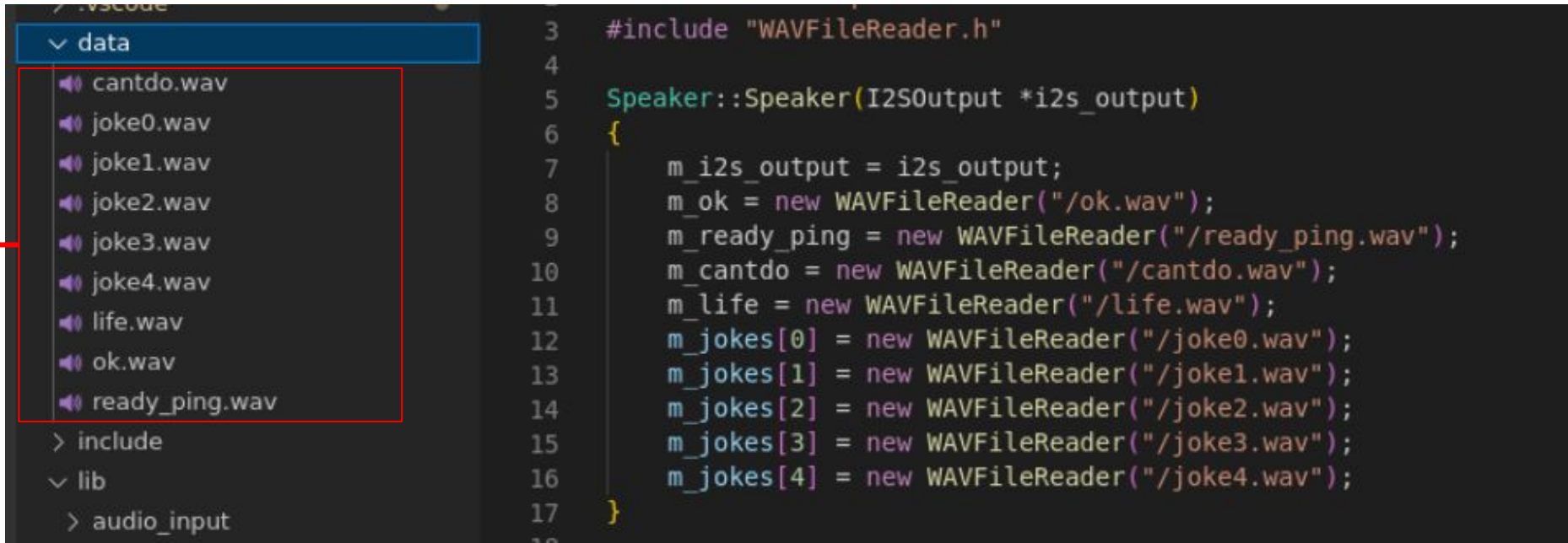


```
1 #include "WAVFileReader.h"
2
3 Speaker::Speaker(I2SOutput *i2s_output)
4 {
5     m_i2s_output = i2s_output;
6     m_ok = new WAVFileReader("/ok.wav");
7     m_ready_ping = new WAVFileReader("/ready_ping.wav");
8     m_cantdo = new WAVFileReader("/cantdo.wav");
9     m_life = new WAVFileReader("/life.wav");
10    m_jokes[0] = new WAVFileReader("/joke0.wav");
11    m_jokes[1] = new WAVFileReader("/joke1.wav");
12    m_jokes[2] = new WAVFileReader("/joke2.wav");
13    m_jokes[3] = new WAVFileReader("/joke3.wav");
14    m_jokes[4] = new WAVFileReader("/joke4.wav");
15 }
16
17
```

File Explorer:

- data
 - cantdo.wav
 - joke0.wav
 - joke1.wav
 - joke2.wav
 - joke3.wav
 - joke4.wav
 - life.wav
 - ok.wav
 - ready_ping.wav
- include
- lib
 - audio_input

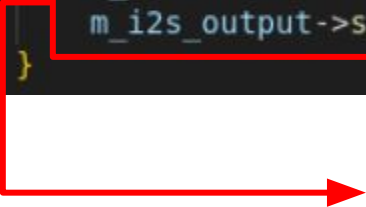
We can play wav file



We can replace those files to whatever we need.

Play the file

```
void Speaker::playOK()  
{  
    m_ok->reset();  
    m_i2s_output->setSampleGenerator(m_ok);  
}
```



Play the wav file

The LED control

```
IndicatorLight.cpp

// This task does all the heavy lifting for our application
void indicatorLedTask(void *param)
{
    IndicatorLight *indicator_light = static_cast<IndicatorLight *>(param);
    const TickType_t xMaxBlockTime = pdMS_TO_TICKS(100);
    while (true)
    {
        // wait for someone to trigger us
        uint32_t ulNotificationValue = ulTaskNotifyTake(pdTRUE, xMaxBlockTime);
        if (ulNotificationValue > 0)
        {
            switch (indicator_light->getState())
            {
            case OFF:
            {
                ledcWrite(0, 0);
                uart2_send("{8701fe}"); // off
                break;
            }
            case ON:
            {
                ledcWrite(0, 255);
                uart2_send("{8701ff}"); // on
                break;
            }
            }
        }
    }
}
```

We can use the `uart2_send` to control the LED.

`{8701fe}` => off

`{8701ff}` => on

color control ???

brightness control ???

Increase model size

<https://github.com/atomic14/voice-controlled-robot/blob/main/model/Train%20Model-All%20Words.ipynb>

- This is an example.
- We increase the classes to be 5. Therefore, the size of dense layer is increased as well.

Layer (type)	Output Shape	Param #
conv_layer1 (Conv2D)	(None, 99, 43, 4)	40
max_pooling1 (MaxPooling2D)	(None, 49, 21, 4)	0
conv_layer2 (Conv2D)	(None, 49, 21, 4)	148
max_pooling3 (MaxPooling2D)	(None, 24, 10, 4)	0
flatten_1 (Flatten)	(None, 960)	0
dropout_2 (Dropout)	(None, 960)	0
hidden_layer1 (Dense)	(None, 80)	76880
dropout_3 (Dropout)	(None, 80)	0
output (Dense)	(None, 5)	405
Total params: 77,473		
Trainable params: 77,473		
Non-trainable params: 0		

Please MP3 file

<https://github.com/atomic14/esp32-play-mp3-demo/blob/main/src/minimp3.h>

- It provide a simple MP3 decoder in a single file.
- We can use MP3 instead of wav to play longer audio file.